

SQL

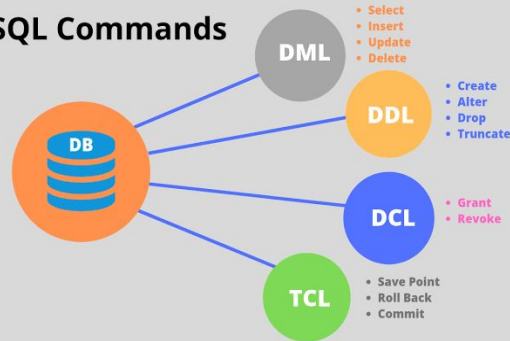
Structured Query Language



SQL Overview



SQL Commands



RDBMS

DDL,DML,DQL

Implementation

What is SQL?



SQL

- A standardized language for relational databases
- Used to define structure and manipulate data
- SQL is a language — not a product

Many relational database management systems implement SQL:

MySQL * PostgreSQL * **SQLite** * SQL Server * MariaDB

RDBMS

(relational database management system)

All of these systems implement SQL, but differ in architecture, features, deployment model and specific syntax (though much is shared)

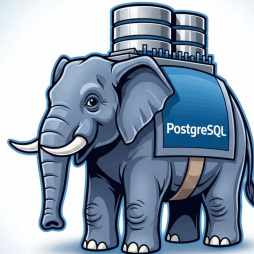
MySQL

An open-source relational database system commonly used for web applications. MySQL is often paired with server-side languages and runs as a database server that multiple applications can connect to simultaneously.



PostgreSQL

An open-source relational database system known for strong standards compliance and advanced features. PostgreSQL is often used when complex data types, strict constraints, or more advanced querying are required.



SQLite

A lightweight, file-based relational database system. SQLite does not require a server and stores the entire database in a single file, making it common in mobile apps, desktop software, and embedded systems.



SQL Server

A relational database system developed by Microsoft. SQL Server is widely used in enterprise environments and integrates closely with the Microsoft ecosystem and tooling.

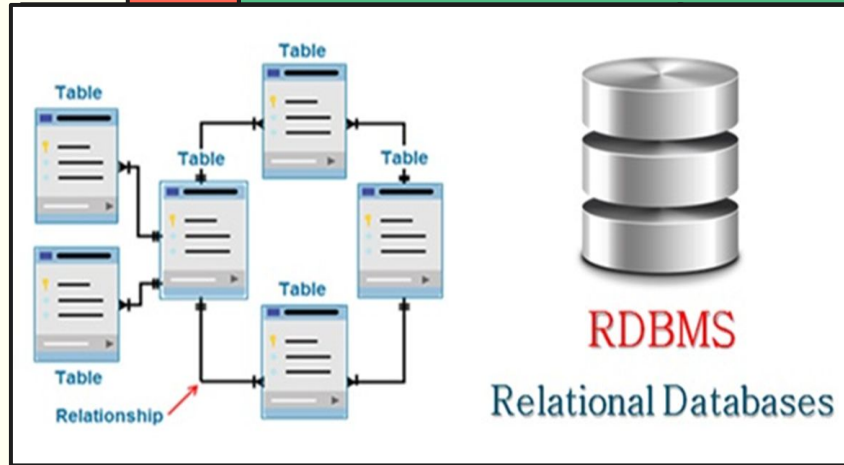


MariaDB

An open-source relational database system that began as a fork of MySQL. MariaDB maintains compatibility with MySQL while remaining community-driven and fully open-source.



What is a RDBMS?



A relational database:

- Stores data in tables
- Organizes data into rows and columns
- Uses keys to define relationships
- Enforces structural rules
- Relational = tables connected through defined relationships.

Core Vocabulary

Table

Collection of related records

Row (Tuple)

One record

Column (Attribute)

Property of a record

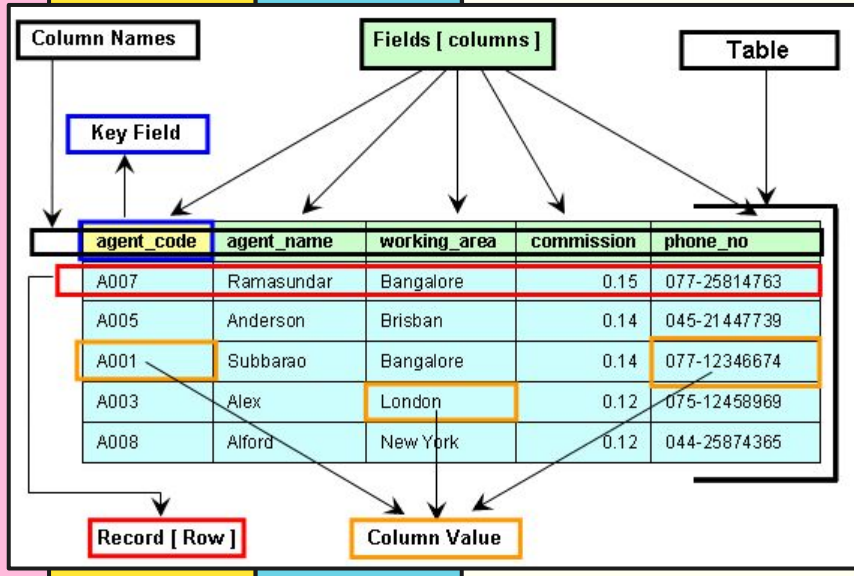
Primary Key

Unique identifier

Foreign Key

Reference to another table

You already designed these in ERDs



What is schema?

Schema = the structural blueprint of the database

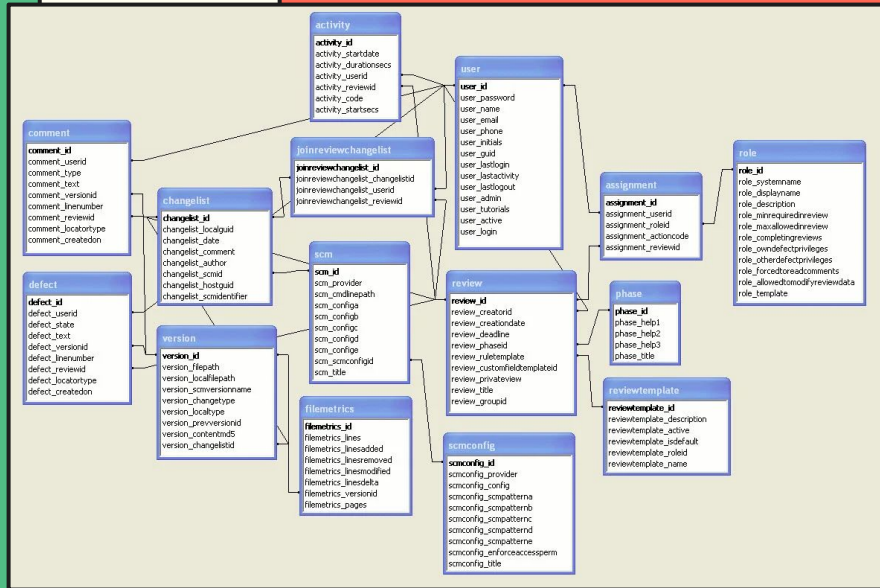
It defines:

- Table names
- Column names
- Data types
- Primary keys
- Foreign keys
- Constraints

Important idea:

The schema enforces structure.

Without a schema, a database is just unstructured storage.



ERD

(logical structure)

SQL



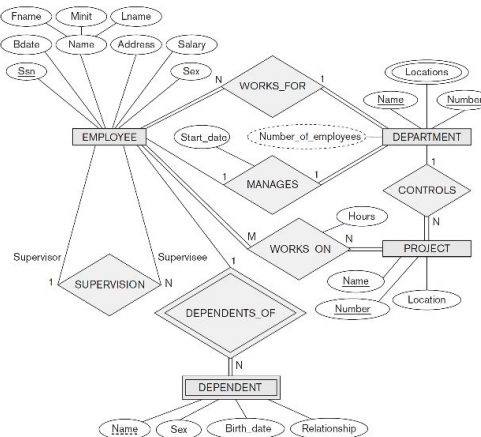
Schema

(implementation)

Requirements

- The company is organized into departments. Each department has a unique name, a unique number, and a particular employee who manages the department. We keep track of the start date when that employee began managing the department. A department may have several locations.
- A department controls a number of projects, each of which has a unique name, a unique number, and a single location.
- The database will store each employee's name, Social Security number,² address, salary, sex (gender), and birth date. An employee is assigned to one department, but may work on several projects, which are not necessarily controlled by the same department. It is required to keep track of the current number of hours per week that an employee works on each project, as well as the direct supervisor of each employee (who is another employee).
- The database will keep track of the dependents of each employee for insurance purposes, including each dependent's first name, sex, birth date, and relationship to the employee.

ER Diagram



Database Schema

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	-----	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
-------	---------	---------	----------------

DEPT_LOCATIONS

Dnumber	Dlocation
---------	-----------

PROJECT

Pname	Pnumber	Plocation	Dnum
-------	---------	-----------	------

WORKS_ON

Essn	Pno	Hours
------	-----	-------

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
------	----------------	-----	-------	--------------

SQL lets us enforce schema at the database level



The rules layer.

**Constraints
protect data
integrity**

Common constraints

- PRIMARY KEY
- FOREIGN KEY
- NOT NULL
- UNIQUE
- CHECK

Examples

- PRIMARY KEY prevents duplicates
- FOREIGN KEY prevents orphan records
- NOT NULL prevents missing required data
- UNIQUE prevents duplicate values in a column (without making it the primary key)
- CHECK enforces custom validation rules on data

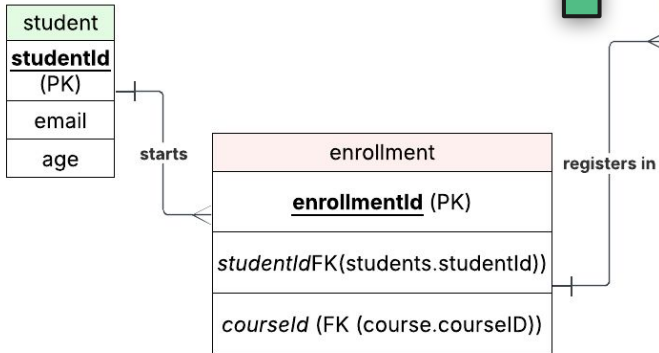
Common SQL Data Types



Numeric Types INTEGER Whole numbers SMALLINT Smaller whole numbers BIGINT Very large whole numbers DECIMAL(p,s) Fixed precision numbers (exact) NUMERIC(p,s) Same as DECIMAL REAL Floating-point number FLOAT Approximate decimal number DOUBLE Double precision float	Text Types CHAR(n) Fixed-length string VARCHAR(n) Variable-length string TEXT Large text data VARCHAR is most common in structured data.	Date/Time Types DATE Calendar date TIME Time of day DATETIME Date + time TIMESTAMP Date + time (often auto-generated)	Boolean Types BOOLEAN True / False (Some systems implement this as INTEGER under the hood.)	Structural Types SERIAL Auto-incrementing integer (PostgreSQL) AUTO_INCREMENT Auto-incrementing integer (MySQL) UUID Universally unique identifier	SQLite officially uses only five storage classes: • NULL • INTEGER • REAL • TEXT • BLOB	SQLite uses type affinity, meaning it is more flexible than MySQL or PostgreSQL. A column has a recommended type (affinity), not a hard rule. SQLite will try to store values using that type — but it won't always reject mismatches if it can map the data onto a type
---	--	--	---	--	---	---

EXAMPLE SCHEMA DEFINITION

ERD



SQLite

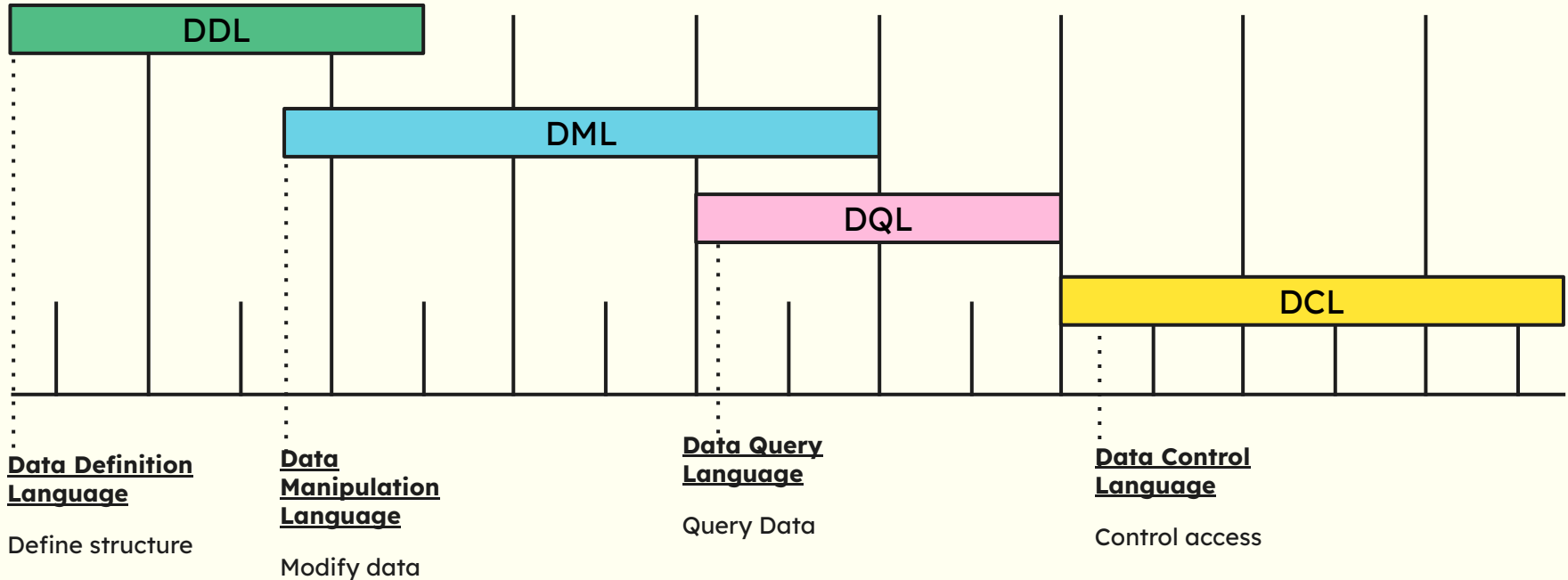
```
PRAGMA foreign_keys = ON;

CREATE TABLE student (
    studentId INTEGER PRIMARY KEY,
    email TEXT UNIQUE NOT NULL,
    age INTEGER CHECK (age >= 0)
);

CREATE TABLE course (
    courseId INTEGER PRIMARY KEY,
    title TEXT NOT NULL
);

CREATE TABLE enrollment (
    enrollmentId INTEGER PRIMARY KEY,
    studentId INTEGER NOT NULL,
    courseId INTEGER NOT NULL,
    FOREIGN KEY (studentId) REFERENCES student(studentId),
    FOREIGN KEY (courseId) REFERENCES course(courseId)
);
```

SQL Command Categories



DDL

Data Definition Language

Defines (creates) database structure.

DDL defines rules and structure.

DDL defines rules and structure.

```
CREATE TABLE student (  
  studentId INTEGER PRIMARY KEY,  
  name TEXT NOT NULL  
);
```

student

studentId
(PK)

name

DML

Data Manipulation Language

Modifies stored data.

Other common DML commands:

- UPDATE
- DELETE

DML operates inside the structure defined by DDL.

```
INSERT INTO student (studentId, name)
VALUES (1, 'Alice');
```

```
sqlite> INSERT INTO student (studentid, name)
...> VALUES(1,Alice);
Parse error: no such column: Alice
INSERT INTO student (studentid, name) VALUES(1,Alice);
error here --^
```

THIS DOES NOT WORK because I do not cast the name as a string. This is what is meant by SQL enforces constraints at the database level

student	
<u>studentId</u> (PK)	name
1	Alice

DQL

Data Query Language

Retrieves data.

SELECT reads data — it does not change it.

```
SELECT name  
FROM student  
WHERE studentId = 1;
```

student	
<u>studentId</u> (PK)	name
1	Alice

```
name  
----  
Alice
```

DCL

Data Control Language

Controls permissions and access.

More common in server databases (MySQL, PostgreSQL).

Not heavily used in SQLite.

Example:
User user1 is allowed to run SELECT queries on the student table.

User user1 would not be allowed to INSERT, UPDATE, DELETE from the student table

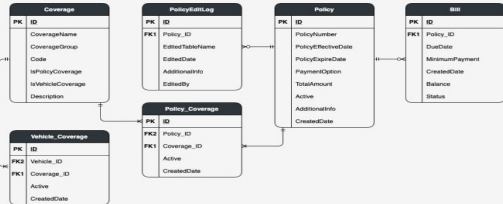
```
GRANT SELECT ON student TO user1;
```

```
SELECT name  
FROM student  
WHERE studentId = 1;
```

student	
<u>studentId</u> (PK)	name
1	Alice

```
name  
-----  
Alice
```

Back-end development



```
SELECT Orders.OrderID, Customers.CustomerName, Orders.OrderDate
FROM Orders
INNER JOIN Customers ON Orders.CustomerID=Customers.CustomerID;
```

```
books.db
create.sql
db_basics.db
demo.sql
streaming_service.db
test2026.db

demo.sql
create.sql
10 author_id INTEGER PRIMARY KEY,
11 name TEXT NOT NULL
12 );
13
14 CREATE TABLE book_authors (
15 book_id INTEGER,
16 author_id INTEGER,
17 PRIMARY KEY (book_id, author_id),
18 FOREIGN KEY (book_id) REFERENCES books(book_id),
19 FOREIGN KEY (author_id) REFERENCES authors(author_id)
20 );
21
```

You have already:

- Designed ERDs
- Identified keys
- Normalized tables
- Resolved many-to-many relationships

Now you are learning:

- Implementing structure with SQL
- Turning logic into enforceable rules
- Moving from diagrams to executable schema

Next step:

- Implement a database in SQLite.